Resolução — Trabalho ARA0363

Programação de Software Básico em C

Questões objetivas (1 a 12)

Questão	Alternativa	Justificativa
1	**B**	A biblioteca windows.h permite criar e gerenciar janelas e controles gráficos.
2	**C**	A classe EDIT cria um campo para entrada de texto.
3	**D**	As constantes identificam controles e permitem tratar eventos.
4	**A**	GetWindowTextA copia o texto do campo para a variável produto.
5	**E**	Rejeita quantidades inválidas, não positivas ou fora do limite.
6	**B**	4 × R$ 75,50 = R$ 302,00.
7	**C**	O modo a acrescenta dados ao fim do arquivo, criando-o se necessário.
8	**A**	O ponto e vírgula separa os campos do registro.
9	**D**	O laço lê linhas sucessivas até não haver mais dados.
10	**E**	sscanf extrai os campos de texto para variáveis.
11	**B**	LB_ADDSTRING adiciona um texto ao componente de lista.
12	**C**	200 + 150 + 800 = R$ 1.150,00; contador = 3.

Questão 13 — Resposta discursiva

A função salvarCompra() recebe os dados da interface, valida as entradas, calcula o total e registra a compra no arquivo aval1234.txt.

Primeiro, GetWindowTextA() obtém o nome do produto. O programa verifica se o nome não está vazio e se não contém caracteres proibidos, como ponto e vírgula e quebras de linha, que poderiam prejudicar a leitura posterior do arquivo.

A função lerQuantidade() converte e valida a quantidade como inteiro positivo dentro do limite definido. A função lerDecimal() converte o preço, aceitando vírgula ou ponto decimal, e verifica se ele é válido e não negativo. Se alguma validação falhar, uma mensagem de erro é apresentada e a compra não é salva.

Em seguida, total = quantidade * preco calcula o valor da compra. A verificação isfinite(total) impede que resultados numéricos inválidos sejam gravados.

O arquivo é aberto com fopen(ARQUIVO, "a"), que permite adicionar novos registros ao final sem apagar os anteriores. fprintf() grava produto, quantidade, preço e total separados por ponto e vírgula. ferror() verifica falhas de escrita e fclose() fecha o arquivo; o retorno do fechamento também é verificado. Se tudo ocorrer corretamente, o programa mostra o total e uma mensagem de sucesso.

Questão 14 — Resposta discursiva

A função mostrarCompras() recupera os registros do arquivo aval1234.txt e os apresenta na lista da interface gráfica.

Inicialmente, fopen(ARQUIVO, "r") abre o arquivo para leitura. Se a abertura falhar, o sistema informa que não há compras cadastradas. A lista é limpa com SendMessageA() usando LB_RESETCONTENT, e as variáveis acumulado e contador começam em zero.

Dentro de um laço while, fgets() lê uma linha por vez. sscanf() interpreta os campos separados por ponto e vírgula, extraindo produto, quantidade, preço e total. Somente linhas com os quatro campos corretamente interpretados são processadas.

A função snprintf() monta um texto legível com os dados de cada compra. SendMessageA() com LB_ADDSTRING insere esse texto no componente de lista. Para cada registro válido, acumulado += total soma o valor da compra e contador++ incrementa o número de compras.

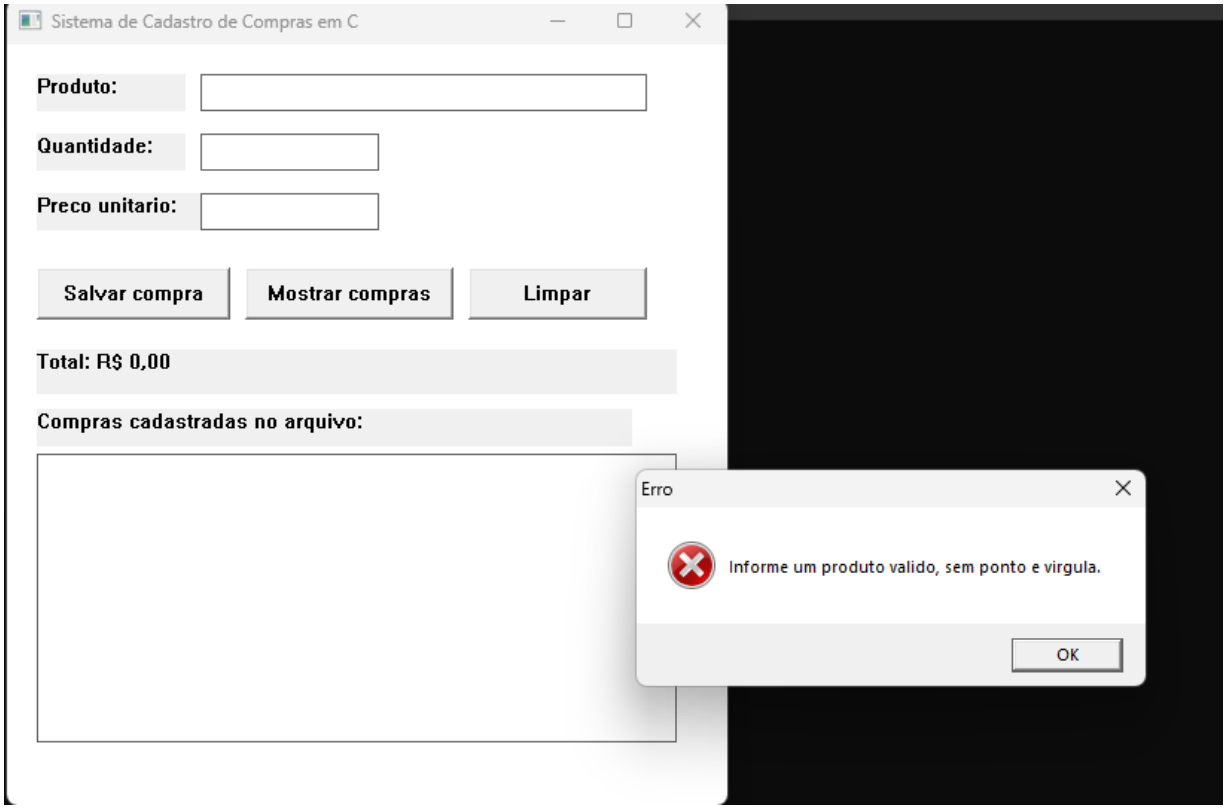
Ao terminar a leitura, fclose() fecha o arquivo. Por fim, SetWindowTextA() apresenta o valor total acumulado e a quantidade de compras exibidas.

Evidências práticas exigidas no enunciado

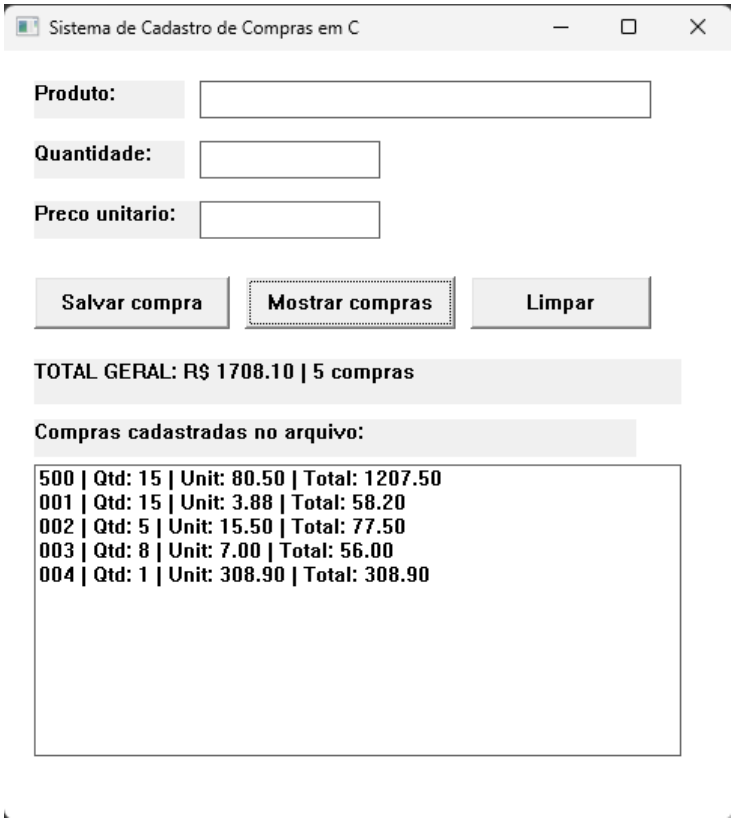
Ainda precisam ser produzidas pelo aluno: captura legível do código-fonte completo no ambiente de desenvolvimento, captura da execução com pelo menos cinco inserções de dados diferentes e link público do repositório GitHub com o código atualizado. Essas evidências não foram fornecidas e não foram simuladas neste documento.

Evidências de execução do programa

Validação de produto inválido



Consulta das cinco compras cadastradas — total geral de R$ 1.708,10

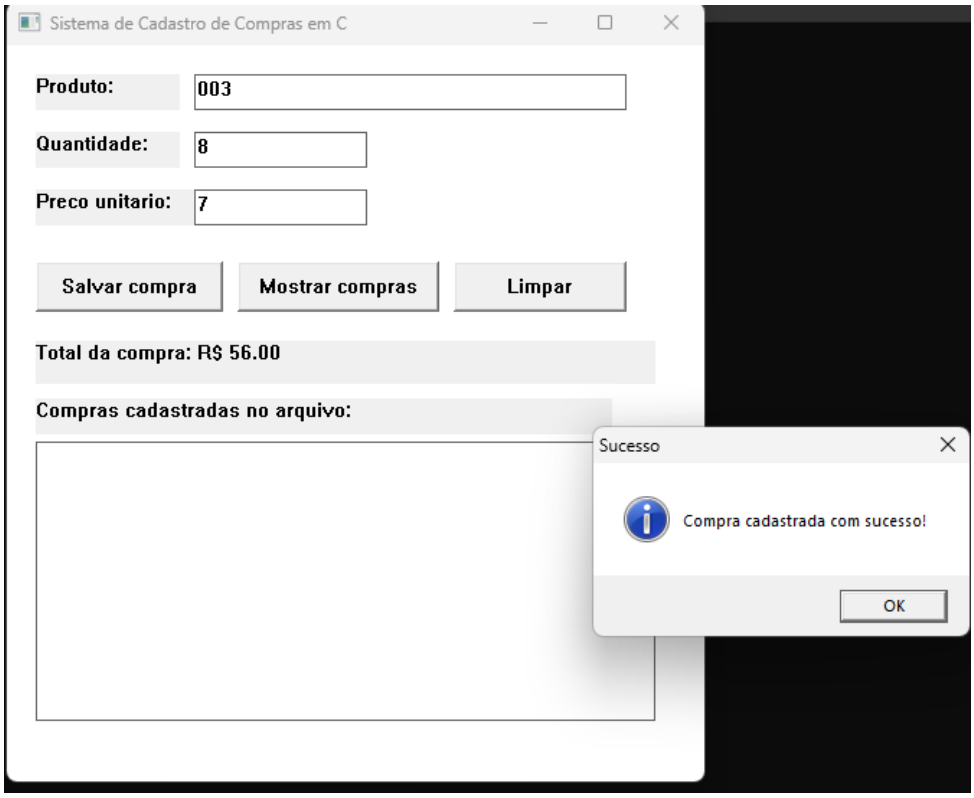


Outros testes do programa

Limpeza dos campos e da lista exibida

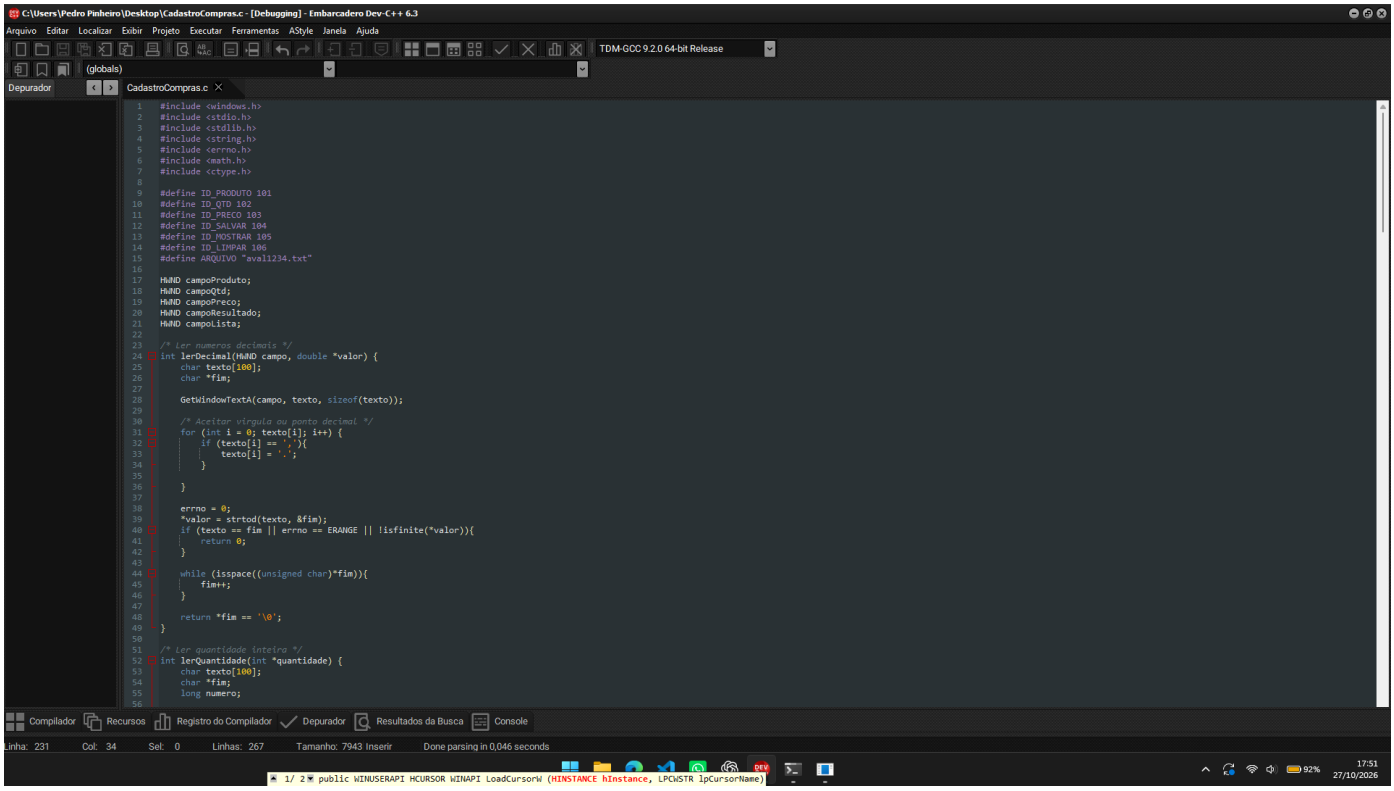


Cadastro realizado com sucesso — produto 003



Capturas do código-fonte

Código-fonte — captura 1 de 5



Código-fonte — captura 2 de 5

CadastroCompras.c

```
    char *fim;
    long numero;

    GetWindowTextA(campoQtd, texto, sizeof(texto));
    errno = 0;
    numero = strtol(texto, &fim, 10);

    if (texto == fim || errno == ERANGE || numero <= 0 || numero > 1000000){
        return 0;
    }

    while (isspace((unsigned char)*fim)){
        fim++;
    }

    if (*fim != '\0'){
        return 0;
    }

    *quantidade = (int)numero;
    return 1;
}

/* Salvar uma compra */
void salvarCompra() {
    char produto[100];
    char mensagem[200];
    int quantidade;
    double preco, total;

    GetWindowTextA(campoProduto, produto, sizeof(produto));

    if (strlen(produto) == 0 || strchr(produto, ';') != NULL || strchr(produto, '\n') != NULL || strchr(produto, '\r') != NULL) {
        MessageBoxA(NULL, "Informe um produto valido, sem ponto e virgula.", "Erro", MB_OK | MB_ICONERROR);

        return;
    }

    if (!lerQuantidade(&quantidade)) {
        MessageBoxA(NULL, "Digite uma quantidade inteira positiva!", "Erro", MB_OK | MB_ICONERROR);

        return;
    }

    if (!lerDecimal(campoPreco, &preco) || preco < 0) {
        MessageBoxA(NULL, "Digite um preco valido!", "Erro", MB_OK | MB_ICONERROR);

        return;
    }

    total = quantidade * preco;

    if (!isfinite(total)) {
        MessageBoxA(NULL, "Total fora do intervalo permitido.", "Erro", MB_OK | MB_ICONERROR);

        return;
```

Registro do Compilador | Depurador | Resultados da Busca | Console

CadastroCompras.c

```c
        return;
    }

    FILE *arquivo = fopen(ARQUIVO, "a");

    if (arquivo == NULL) {
        MessageBoxA(NULL, "Nao foi possivel abrir o arquivo!", "Erro", MB_OK | MB_ICONERROR);

        return;
    }

    /* Grava os dados separados por: */
    int gravou = fprintf(arquivo, "%s;%d;%.2f;%.2f\n", produto, quantidade, preco, total);
    int erro = ferror(arquivo);
    int fechou = fclose(arquivo);

    if (gravou < 0 || erro || fechou != 0) {
        MessageBoxA(NULL, "Erro ao gravar a compra!", "Erro", MB_OK | MB_ICONERROR);

        return;
    }

    snprintf(mensagem, sizeof(mensagem), "Total da compra: R$ %.2f", total);
    SetWindowTextA(campoResultado, mensagem);
    MessageBoxA(NULL, "Compra cadastrada com sucesso!", "Sucesso", MB_OK | MB_ICONINFORMATION);
}

/* Mostrar todas as compras do arquivo. */
void mostrarCompras() {
    FILE *arquivo = fopen(ARQUIVO, "r");

    if (arquivo == NULL) {
        MessageBoxA(NULL, "Nenhuma compra cadastrada!", "Aviso", MB_OK | MB_ICONINFORMATION);

        return;
    }

    char linha[300];
    char produto[100];
    char texto[500];
    int quantidade;
    double preco, total;
    double acumulado = 0;
    int contador = 0;

    /* Limpar a lista */
    SendMessageA(campoLista, LB_RESETCONTENT, 0, 0);

    while (fgets(linha, sizeof(linha), arquivo)) {
        if (sscanf(linha, "%99[^;];%d;%lf;%lf", produto, &quantidade, &preco, &total) == 4) {
            snprintf(texto, sizeof(texto), "%s | Qtd: %d | Unit: %.2f | Total: %.2f", produto, quantidade, preco, total);
            SendMessageA(campoLista, LB_ADDSTRING, 0, (LPARAM)texto);
            acumulado += total;

            contador++;
        }
```

Código-fonte — captura 4 de 5

CadastroCompras.c

```
            contador++;
        }
    }

    fclose(arquivo);
    snprintf(texto, sizeof(texto), "TOTAL GERAL: R$ %.2f | %d compras", acumulado, contador);
    SetWindowTextA(campoResultado, texto);
}

/* Limpar campos. */
void limparCampos() {
    SetWindowTextA(campoProduto, "");
    SetWindowTextA(campoQtd, "");
    SetWindowTextA(campoPreco, "");
    SetWindowTextA(campoResultado, "Total: R$ 0,00");
    SendMessageA(campoLista, LB_RESETCONTENT, 0, 0);
}

/* Funcao principal da janela */
LRESULT CALLBACK Janela(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {

    switch (msg) {
    case WM_CREATE:
        CreateWindowA("STATIC", "Produto:", WS_VISIBLE | WS_CHILD, 20, 20, 100, 25, hwnd, NULL, NULL, NULL);
        campoProduto = CreateWindowA("EDIT", "", WS_VISIBLE | WS_CHILD | WS_BORDER | ES_AUTOHSCROLL, 130, 20, 300, 25, hwnd, (HMENU)ID_PRODUTO, NULL, NULL);
        CreateWindowA("STATIC", "Quantidade:", WS_VISIBLE | WS_CHILD, 20, 60, 100, 25, hwnd, NULL, NULL, NULL);
        campoQtd = CreateWindowA("EDIT", "", WS_VISIBLE | WS_CHILD | WS_BORDER | ES_AUTOHSCROLL, 130, 60, 120, 25, hwnd, (HMENU)ID_QTD, NULL, NULL);
        CreateWindowA("STATIC", "Preco unitario:", WS_VISIBLE | WS_CHILD, 20, 100, 110, 25, hwnd, NULL, NULL, NULL);
        campoPreco = CreateWindowA("EDIT", "", WS_VISIBLE | WS_CHILD | WS_BORDER | ES_AUTOHSCROLL, 130, 100, 120, 25, hwnd, (HMENU)ID_PRECO, NULL, NULL);
        CreateWindowA("BUTTON", "Salvar compra", WS_VISIBLE | WS_CHILD, 20, 150, 130, 35, hwnd, (HMENU)ID_SALVAR, NULL, NULL);
        CreateWindowA("BUTTON", "Mostrar compras", WS_VISIBLE | WS_CHILD, 160, 150, 140, 35, hwnd, (HMENU)ID_MOSTRAR, NULL, NULL);
        CreateWindowA("BUTTON", "Limpar", WS_VISIBLE | WS_CHILD, 310, 150, 120, 35, hwnd, (HMENU)ID_LIMPAR, NULL, NULL);
        campoResultado = CreateWindowA("STATIC", "Total: R$ 0,00", WS_VISIBLE | WS_CHILD, 20, 205, 430, 30, hwnd, NULL, NULL, NULL);
        CreateWindowA("STATIC", "Compras cadastradas no arquivo:", WS_VISIBLE | WS_CHILD, 20, 245, 400, 25, hwnd, NULL, NULL, NULL);
        campoLista = CreateWindowA("LISTBOX", "", WS_VISIBLE | WS_CHILD | WS_BORDER | WS_VSCROLL | LBS_NOTIFY, 20, 275, 430, 200, hwnd, NULL, NULL, NULL);
        break;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
            case ID_SALVAR:
                salvarCompra();
                break;
            case ID_MOSTRAR:
                mostrarCompras();
                break;
            case ID_LIMPAR:
                limparCampos();
                break;
        }
        break;

    case WM_DESTROY:
        PostQuitMessage(0);
        break;
    default:
        return
```

Código-fonte — captura 5 de 5

CadastroCompras.c

```c
                mostrarCompras();
                break;
            case ID_LIMPAR:
                limparCampos();
                break;
        }
        break;

    case WM_DESTROY:
        PostQuitMessage(0);
        break;
    default:
        return
        DefWindowProcA(hwnd, msg, wParam, lParam);
    }
    return 0;
}

/* Funcao principal do Windows */
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nCmdShow) {
        WNDCLASSA wc = {0};
        wc.lpfnWndProc = Janela;
        wc.hInstance = hInstance;
        wc.lpszClassName = "CadastroCompras";
        wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
        wc.hCursor = LoadCursor(NULL, IDC_ARROW);

        if (!RegisterClassA(&wc)){
            return 1;
        }

        HWND hwnd = CreateWindowA("CadastroCompras", "Sistema de Cadastro de Compras em C", WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT, 500, 550, NULL, NULL, hInstance, NULL);

        if (!hwnd){
            return 1;
        }

        ShowWindow(hwnd, nCmdShow);
        UpdateWindow(hwnd);

        MSG msg;

        while (GetMessage(&msg, NULL, 0, 0) > 0) {
            TranslateMessage(&msg);
            DispatchMessageA(&msg);
    }

    return (int)msg.wParam;
}
```

Repositório do código-fonte

https://github.com/PedroShinro/Backend-de-baixo-nivel